



UNIVERSIDAD DE LOS LLANOS
Facultad de Ciencias básicas e ingenierías

INFORME DE LABORATORIO OPTIMIZACIÓN

Teoría de Grafos

Sebastian Obando Rojas ¹

1. Cod: 160004526 Ing. Sistemas
Facultad de Ciencias Básicas e Ingenierías.

Resumen

Este laboratorio implementa la Teoría de Grafos para representar, analizar y visualizar grafos dirigidos y no dirigidos mediante una arquitectura Python modular de tres capas: el núcleo matemático (grafo.py), la interfaz gráfica (interfaz.py) y el punto de entrada (main.py). La aplicación permite al usuario configurar el número de vértices y el tipo de grafo, ingresar la matriz de adyacencia de forma interactiva, y obtener automáticamente la representación matemática, la lista de adyacencia, los grados de los nodos y la visualización gráfica.

El sistema cubre los conceptos fundamentales exigidos por la guía: detección de caminos mediante BFS, identificación de ciclos mediante DFS, nodos adyacentes y verificación de conectividad fuerte. La representación gráfica se dibuja en tiempo real sobre un Canvas de Tkinter, resaltando con colores diferenciados los caminos encontrados (amarillo) y los ciclos detectados (rojo).

Los conceptos clave implementados incluyen: construcción de la matriz de adyacencia cuadrada $n \times n$, representación matemática del grafo como $V = \{v_1, \dots, v_n\}$ y $A = \{<v_i, v_j>, \dots\}$, búsqueda en amplitud (BFS) para caminos, búsqueda en profundidad (DFS) con coloreo para ciclos, y verificación de conectividad mediante recorrido en la transpuesta del grafo. La interfaz fue desarrollada en Tkinter estándar con diseño interactivo por clic sobre la matriz.

Palabras clave: Teoría de Grafos, Matriz de Adyacencia, BFS, DFS, Caminos, Ciclos, Nodos Adyacentes, Python, Tkinter.

1. Introducción

La Teoría de Grafos es una rama fundamental de las matemáticas discretas y la informática, ampliamente utilizada en Investigación de Operaciones para modelar redes, relaciones y flujos. Un grafo $G = (V, A)$ está conformado por un conjunto de vértices V y un conjunto de aristas A que los conectan; dependiendo de si las aristas tienen orientación o no, se habla de grafos dirigidos (digrafos) o no dirigidos.

En entornos reales (redes de transporte, redes sociales, circuitos eléctricos, optimización de procesos) los grafos pueden tener cientos de nodos y miles de aristas, lo que hace indispensable contar con herramientas computacionales que automaticen su representación y análisis. Este laboratorio responde a esa necesidad construyendo una solución completa que combina:

- Un motor matemático robusto (grafo.py) que implementa la matriz de adyacencia, la representación formal, los recorridos BFS/DFS y la verificación de conectividad.
- Una interfaz gráfica interactiva (interfaz.py) con entrada de datos por clic, canvas de dibujo en tiempo real y panel de resultados desplazable.

- Una arquitectura limpia dividida en módulos con responsabilidades claras y un punto de entrada unificado (main.py).

El objetivo es comprender tanto los conceptos matemáticos del grafo como su implementación en código mantenible, con visualización que facilite la comprensión de caminos, ciclos y adyacencias, competencias esenciales en ingeniería de sistemas aplicada a la optimización.

2. Referente teórico

Teoría de Grafos:

Un grafo $G = (V, A)$ está compuesto por un conjunto no vacío de vértices V y un conjunto de aristas A , donde cada arista conecta dos vértices. Existen dos tipos principales: grafos no dirigidos, en que $(v, w) = (w, v)$, y grafos dirigidos (digrafos), en que $\langle v, w \rangle \neq \langle w, v \rangle$. En los digrafos, v es la cola y w es la cabeza de la arista.

Representación mediante Matriz de Adyacencia:

Una matriz cuadrada M de orden $n \times n$ donde el elemento $M[i][j] = 1$ si existe una arista entre el nodo i y el nodo j , y 0 en caso contrario. Para grafos no dirigidos la matriz es simétrica. La diagonal principal siempre vale 0 (no se permiten bucles).

Lista de Adyacencia:

Para cada nodo v se mantiene la lista de todos los nodos w tales que existe la arista $\langle v, w \rangle$. Es una representación más compacta que la matriz cuando el grafo es disperso.

Camino y Ciclo:

Un camino es una secuencia de vértices $v_1, v_2, \dots, v_k$ tal que existe arista entre vértices consecutivos. Su longitud es $k-1$ (número de aristas). Un ciclo es un camino en que el primer y el último vértice coinciden; se llama ciclo simple si los demás vértices son distintos.

Nodos Adyacentes:

En un grafo dirigido, los nodos adyacentes a v son todos los w con $\langle v, w \rangle \in A$ (adyacentes hacia afuera) y los u con $\langle u, v \rangle \in A$ (adyacentes hacia adentro). En un grafo no dirigido, todos los nodos unidos a v mediante cualquier arista.

Grafo Conexo:

Existe un camino entre cualquier par de vértices. En grafos dirigidos se habla de grafo fuertemente conexo cuando esto se cumple en ambas direcciones (alcanzabilidad en el grafo original y en su transpuesta).

BFS (Búsqueda Primero en Amplitud):

Explora el grafo nivel por nivel usando una cola. Visita primero el nodo inicial, luego todos sus adyacentes, después los adyacentes de éstos, y así sucesivamente. Garantiza encontrar el camino más corto en grafos no ponderados.

DFS (Búsqueda Primero en Profundidad):

Explora tan lejos como sea posible a lo largo de cada rama antes de retroceder. Se implementa con coloreo de nodos (blanco=no visitado, gris=en proceso, negro=completado) para detectar aristas de retroceso y, con ellas, la existencia de ciclos.

Arquitectura del software:

Concepto	Definición breve	Aplicación en el Lab
Python 3	Lenguaje de propósito general con tipado dinámico	Núcleo del algoritmo y lógica de la GUI
Tkinter	Biblioteca estándar de GUI para Python	Construcción de la interfaz gráfica y Canvas
collections.deque	Cola de doble extremo de la stdlib	Implementación eficiente de BFS
Clase Grafo	Encapsula la matriz y los algoritmos	Separación entre lógica y presentación
DFS con coloreo	Detección de ciclos con nodos blanco/gris/negro	Identificar el primer ciclo en el grafo
BFS desde origen	Camino más corto no ponderado	Mostrar camino entre v1 y v2 en la GUI
Transpuesta del grafo	Invertir la dirección de todas las aristas	Verificar conectividad fuerte en digrafos

3. Procedimiento

3.1 Construcción de la Clase Grafo y la Matriz de Adyacencia

La clase Grafo recibe el número de vértices n y un indicador de tipo (dirigido o no dirigido). Inicializa una matriz $n \times n$ en cero y provee métodos para agregar, eliminar y alternar aristas.

Archivo: grafo.py

```
class Grafo:
    def __init__(self, n: int, dirigido: bool = True):
        self.n = n
        self.dirigido = dirigido
        self.matriz = [[0] * n for _ in range(n)]

    def agregar_arista(self, i: int, j: int):
        """Agrega la arista (i, j). Si no es dirigido, también (j, i)."""
        if i == j:
            return # La diagonal siempre queda en 0
        self.matriz[i][j] = 1
        if not self.dirigido:
            self.matriz[j][i] = 1

    def toggle_arista(self, i: int, j: int):
        """Alterna la existencia de la arista (i, j)."""
        if i == j:
            return
        if self.matriz[i][j] == 0:
            self.agregar_arista(i, j)
        else:
            self.eliminar_arista(i, j)
```

3.2 Representación Matemática del Grafo

Los métodos `representacion_V` y `representacion_A` generan la notación formal del grafo. Para grafos dirigidos, las aristas se escriben como pares ordenados $\langle v_i, v_j \rangle$; para no dirigidos, como conjuntos $\{v_i, v_j\}$.

Archivo: *grafo.py*

```
def representacion_V(self) -> str:
    nombres = self.nombres_vertices()
    return "V = {" + ", ".join(nombres) + "}"

def representacion_A(self) -> str:
    nombres = self.nombres_vertices()
    aristas = []
    if self.dirigido:
        for i in range(self.n):
            for j in range(self.n):
                if self.matriz[i][j]:
                    aristas.append(f"<{nombres[i]}, {nombres[j]}>")
    else:
        visitadas = set()
        for i in range(self.n):
            for j in range(self.n):
                if self.matriz[i][j] and (j, i) not in visitadas:
                    aristas.append(f"{{ {nombres[i]}, {nombres[j]} }}")
                    visitadas.add((i, j))
    return "A = {" + ", ".join(aristas) + "}"
```

3.3 Lista de Adyacencia y Grados de los Nodos

El método `lista_adyacencia` construye un diccionario donde cada clave es un vértice y su valor es la lista de vecinos. El método `grados` devuelve, para grafos dirigidos, el grado de entrada y salida de cada nodo; para no dirigidos, el grado total.

Archivo: *grafo.py*

```
def lista_adyacencia(self) -> dict[str, list[str]]:
    nombres = self.nombres_vertices()
    result = {}
    for i in range(self.n):
        vecinos = [nombres[j] for j in range(self.n) if self.matriz[i][j]]
        result[nombres[i]] = vecinos
    return result

def grados(self) -> dict[str, dict]:
    nombres = self.nombres_vertices()
    result = {}
    for i in range(self.n):
        if self.dirigido:
```

```

        entrada = sum(self.matriz[j][i] for j in range(self.n))
        salida  = sum(self.matriz[i][j] for j in range(self.n))
        result[nombres[i]] = {"entrada": entrada, "salida": salida}
    else:
        grado = sum(self.matriz[i][j] for j in range(self.n))
        result[nombres[i]] = {"grado": grado}
    return result

```

3.4 Camino — *BFS desde Origen hasta Destino*

El método `encontrar_camino` implementa BFS desde el vértice origen hasta el destino. Usa una cola (deque) y un arreglo padre para reconstruir el camino al finalizar. Devuelve la lista de vértices del camino o `None` si no existe conexión.

Archivo: *grafo.py*

```

def encontrar_camino(self, origen: int = 0, destino: int = 1) -> list[str] | None:
    if origen == destino:
        return [self.nombres_vertices()[origen]]
    visitado = [False] * self.n
    padre    = [-1] * self.n
    cola = deque([origen])
    visitado[origen] = True
    while cola:
        actual = cola.popleft()
        for vecino in range(self.n):
            if self.matriz[actual][vecino] and not visitado[vecino]:
                visitado[vecino] = True
                padre[vecino] = actual
                if vecino == destino:
                    return self._reconstruir_camino(padre, origen, destino)
                cola.append(vecino)
    return None

def _reconstruir_camino(self, padre, origen, destino) -> list[str]:
    nombres = self.nombres_vertices()
    camino, actual = [], destino
    while actual != -1:
        camino.append(nombres[actual])
        actual = padre[actual]
    return camino[::-1]

```

3.5 Ciclo — *DFS con Coloreo de Nodos*

El método `encontrar_ciclo` implementa DFS con tres colores: blanco (0 = no visitado), gris (1 = en proceso) y negro (2 = completado). Cuando se detecta una arista hacia un nodo gris, se ha encontrado una arista de retroceso que implica la existencia de un ciclo. El ciclo se reconstruye siguiendo el arreglo padre.

Archivo: grafo.py

```
def encontrar_ciclo(self) -> list[str] | None:
    color = [0] * self.n # 0=blanco, 1=gris, 2=negro
    padre = [-1] * self.n
    ciclo_inicio = [None]; ciclo_fin = [None]

    def dfs(u):
        color[u] = 1
        for v in range(self.n):
            if self.matriz[u][v]:
                if color[v] == 1: # arista de retroceso -> ciclo
                    ciclo_inicio[0] = v; ciclo_fin[0] = u
                    return True
                if color[v] == 0:
                    padre[v] = u
                    if dfs(v): return True
        color[u] = 2
        return False

    for s in range(self.n):
        if color[s] == 0:
            if dfs(s): break

    if ciclo_inicio[0] is None: return None
    nombres = self.nombres_vertices()
    camino, actual = [], ciclo_fin[0]
    while actual != ciclo_inicio[0]:
        camino.append(nombres[actual])
        actual = padre[actual]
    camino.append(nombres[ciclo_inicio[0]])
    camino.reverse()
    camino.append(nombres[ciclo_inicio[0]]) # cierra el ciclo
    return camino
```

3.6 Verificación de Conectividad Fuerte

El método `es_fuertemente_conexo` verifica si el grafo es conexo. Para grafos no dirigidos basta un BFS desde el nodo 0. Para grafos dirigidos se realiza un primer BFS en el grafo original y luego un segundo BFS en la matriz transpuesta; si en ambos casos se alcanzan todos los nodos, el grafo es fuertemente conexo.

Archivo: grafo.py

```
def es_fuertemente_conexo(self) -> bool:
    if self.n == 0: return True

    def alcanzables(src, mat):
        vis = [False] * self.n
        cola = deque([src])
        vis[src] = True
        while cola:
```

```

        u = cola.popleft()
        for v in range(self.n):
            if mat[u][v] and not vis[v]:
                vis[v] = True; cola.append(v)
        return all(vis)

    if not self.dirigido:
        return alcanzables(0, self.matriz)
    if not alcanzables(0, self.matriz): return False
    transpuesta = [[self.matriz[j][i] for j in range(self.n)]
                    for i in range(self.n)]
    return alcanzables(0, transpuesta)

```

3.7 Interfaz Gráfica — Visualización del Grafo

La clase App hereda de tk.Tk y organiza la interfaz en tres columnas: (0) panel de configuración y matriz interactiva, (1) canvas de dibujo del grafo, y (2) panel de resultados con scroll. Cada clic sobre una celda de la matriz alterna la arista correspondiente y redibuja el grafo en tiempo real.

El método `_draw` posiciona los vértices en círculo, dibuja las aristas con flechas para digrafos, y resalta en amarillo el camino hallado y en rojo el ciclo detectado.

Archivo: *interfaz.py*

```

def _draw(self):
    cv.delete('all')
    # Posiciones en círculo
    pos = [(cx + r*cos(-90 + i*360/n), cy + r*sin(-90 + i*360/n))
           for i in range(n)]
    # Color de cada arista
    def ecol(i, j):
        ni, nj = nms[i], nms[j]
        if (ni, nj) in cic_set: return ROJO
        if (ni, nj) in cam_set: return AMARILLO
        return ACENTO
    # Dibujar aristas
    for i in range(n):
        for j in range(n):
            if not g.matriz[i][j]: continue
            if not g.dirigido and j < i: continue
            col = ecol(i, j)
            if g.dirigido:
                cv.create_line(sx,sy, ex,ey, fill=col,
                              arrow=tk.LAST, arrowshape=(10,12,4))
            else:
                cv.create_line(sx,sy, ex,ey, fill=col)
    # Dibujar nodos
    for i, (x, y) in enumerate(pos):
        cv.create_oval(x-R,y-R, x+R,y+R, fill=ACENTO)
        cv.create_text(x, y, text=nms[i], fill='white')

```

3.8 Punto de Entrada — main.py

El archivo main.py es el punto de entrada de la aplicación. Importa la clase App desde interfaz.py e invoca su ciclo principal.

Archivo: main.py

```
from interfaz import App

def main():
    app = App()
    app.mainloop()

if __name__ == "__main__":
    main()
```

4. Resultados

4.1 Ejemplo de Ejecución

Para ilustrar el funcionamiento se configura un grafo dirigido de 5 vértices con las siguientes aristas definidas mediante la matriz de adyacencia interactiva:

	v1	v2	v3	v4	v5
v1	0	0	0	0	0
v2	0	0	0	0	0
v3	0	0	0	0	0
v4	0	0	0	0	0
v5	0	0	0	0	0

Figura 1. La representación matemática generada automáticamente es:

$V = \{v1, v2, v3, v4, v5\}$

$A = \{<v1,v2>, <v2,v3>, <v3,v4>, <v3,v5>, <v5,v1>, <v5,v4>\}$

4.2 Visualización en la Interfaz

Al iniciar la aplicación se presenta, donde el usuario especifica el tamaño $n \times n$ y el tipo de grafo (dirigido o no dirigido). Al pulsar 'Crear matriz' aparece la matriz interactiva, el canvas y el panel de resultados.

The screenshot shows the initial configuration screen of the application. At the top, there is a blue header with the text 'Teoria de Grafos - Practica N 04' and 'Universidad de Los Llanos | Ingenieria de Sistemas | Optimizacion'. Below the header, there is a section titled 'PASO 1 - CONFIGURAR EL GRAFO'. In this section, there is a label 'Tamano n x n:' followed by a text input field containing the number '3'. To the right of this is a label 'Tipo:' followed by a dropdown menu showing 'Dirigido'. To the right of the dropdown are two buttons: 'Crear matriz' (highlighted in blue) and 'Limpiar' (white with a black border).

Figura 2. Configuración inicial.

The screenshot shows the 'PASO 2 - ARISTAS (clic=1, diagonal=0)' section. It features a 5x5 grid representing an adjacency matrix for 5 vertices (v1 to v5). The grid is as follows:

	v1	v2	v3	v4	v5
v1	0	0	0	0	0
v2	0	0	1	0	0
v3	0	0	0	0	0
v4	0	0	0	0	0
v5	0	1	0	0	0

Below the grid, there are two buttons: 'Calcular' (highlighted in blue) and 'Limpiar todo' (white with a black border). To the right of these buttons, the text '2 aristas' is displayed.

Figura 3. Matriz interactiva de adyacencia

Cada celda de la matriz puede activarse (valor 1, fondo Azul) o desactivarse (valor 0, fondo gris claro) con un clic. La diagonal permanece fija en 0. El contador de aristas se actualiza en tiempo real junto con la representación gráfica en el canvas central.

4.3 Visualización del grafo

A medida que se seleccionan las aristas, el grafo aparece en tiempo real en la sección de grafo.

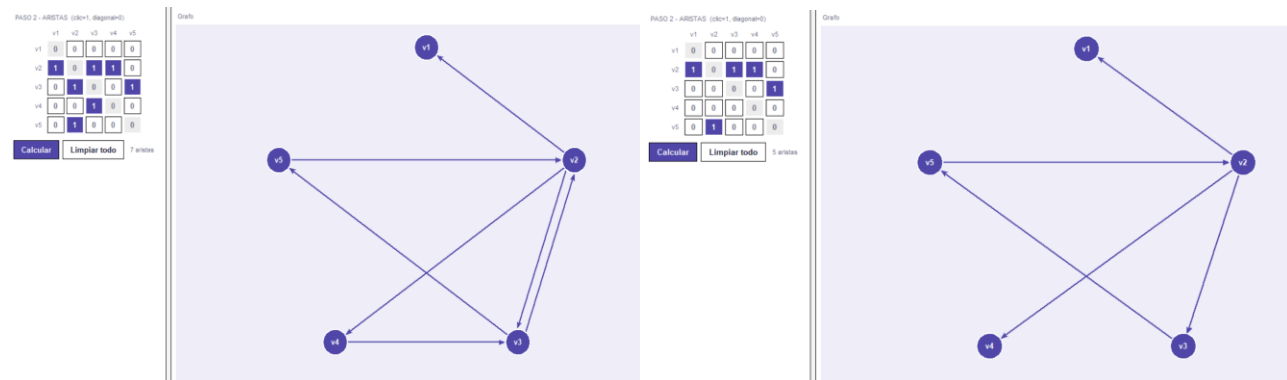


Figura 4. Podemos verificar el grafo en tiempo real, en este caso es direccional, se pueden observar las direcciones de cada arista seleccionada.

4.3 Resultados del Cálculo

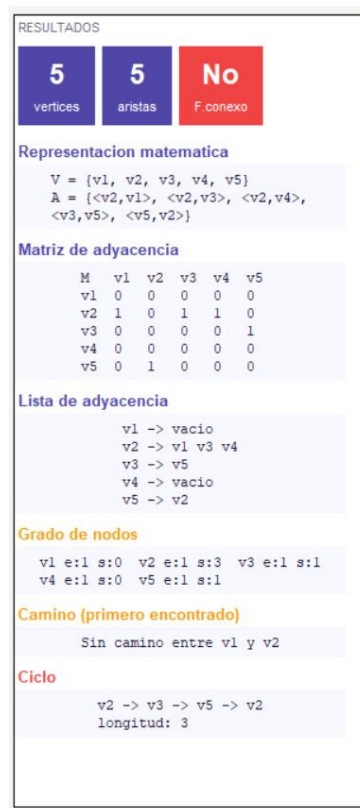


Figura 5. Resultado al momento de oprimir la opción calcular.

Al pulsar el botón 'Calcular' como lo muestra la **Figura 5**, la aplicación ejecuta los algoritmos BFS y DFS y muestra en el panel derecho los siguientes bloques de información:

- **Estadísticas:** número de vértices, número de aristas y estado de conectividad (fuertemente conexo: Sí/No).
- **Representación matemática:** conjunto V y conjunto A con notación formal.
- **Matriz de adyacencia:** tabla alineada con encabezados de columna y fila.

- **Lista de adyacencia:** para cada vértice, sus vecinos directos.
- **Grado de nodos:** grado de entrada y salida (digrafos) o grado total (no dirigidos).
- **Camino (primero encontrado):** secuencia $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ con su longitud.
- **Ciclo detectado:** secuencia que cierra en el nodo inicial, o 'Sin ciclos detectados'.

4.4 Resultado Visual de Caminos y Ciclos

La representación gráfica distingue visualmente los elementos del grafo: las aristas normales, las aristas que forman el camino encontrado, y las aristas que componen el ciclo detectado. Los vértices se posicionan uniformemente en círculo sobre el canvas.

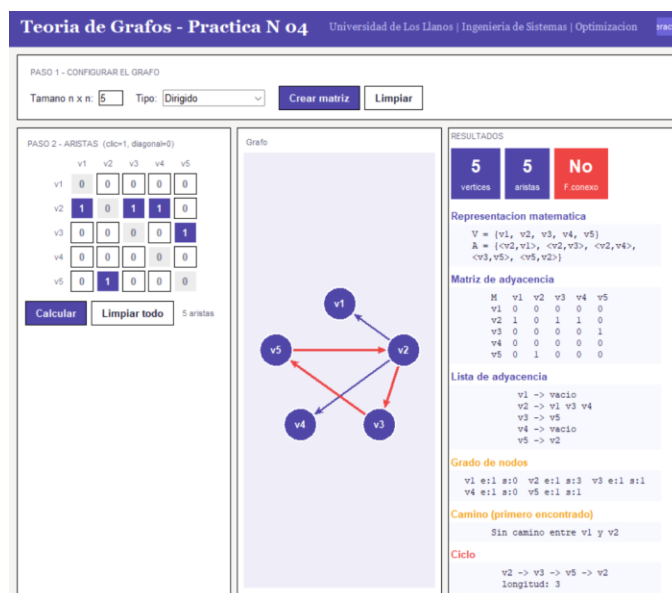


Figura 6. Vista del canvas con ciclo resaltado y camino.

5. Conclusiones

Este laboratorio permitió consolidar tanto el conocimiento matemático de la Teoría de Grafos como las competencias de desarrollo de sistemas necesarias para implementarla y visualizarla de forma interactiva.

La matriz de adyacencia es una representación intuitiva y eficiente para grafos densos: su implementación como lista de listas en Python permite acceso $O(1)$ a cualquier arista y facilita la iteración sobre los vecinos de un nodo mediante un simple recorrido de fila o columna.

La separación entre el módulo de lógica (grafo.py) y el módulo de presentación (interfaz.py) es clave para la mantenibilidad del sistema. El módulo de lógica puede ser probado de forma independiente y reutilizado en otros contextos sin modificar la interfaz gráfica.

El algoritmo BFS garantiza encontrar el camino más corto entre dos vértices en grafos no ponderados, lo que lo convierte en la elección natural para esta tarea. La reconstrucción del camino mediante el arreglo padre permite recuperar la secuencia completa de vértices con un único recorrido hacia atrás desde el destino hasta el origen.

El DFS con coloreo tricolor (blanco/gris/negro) es una técnica elegante para detectar ciclos en grafos dirigidos: la presencia de una arista hacia un nodo gris (en proceso en la pila de recursión) es condición suficiente para afirmar la existencia de un ciclo. En grafos no dirigidos bastaría con marcar los nodos visitados, pero el coloreo tricolor generaliza correctamente para digrafos.

La verificación de conectividad fuerte mediante dos BFS (uno en el grafo original y otro en su transpuesta) demuestra cómo una propiedad global del grafo puede comprobarse eficientemente mediante recorridos locales desde un único nodo fuente.

La visualización en tiempo real sobre el canvas de Tkinter añade un valor pedagógico significativo: al poder alternar aristas con un solo clic y observar inmediatamente el impacto en caminos, ciclos y conectividad, el usuario desarrolla una intuición geométrica del grafo que complementa la comprensión algebraica de la matriz de adyacencia.

6. Bibliografía

[1] Universidad de los Llanos, "Guía 3: Teoría de Grafos," Laboratorio de Optimización, Ingeniería de Sistemas, 2025.